Search

Các tiêu chí đánh giá các chiến lược tìm kiếm (search strategies)

- Một chiến lược tìm kiếm được định nghĩa bằng chọn thứ tự của việc mở rộng nút
- độ hoàn chỉnh (Completeness): Có phải lúc nào cũng tìm ra nghiệm nếu có không?
- độ phức tạp về thời gian (Time complexity): số lượng các trạng thái đã đến
- độ phức tạp về không gian (Space complexity): số lượng các trạng thái tối đa phải lưu trong bộ nhớ
- độ tối ưu (Optimality): có phải lúc nào cũng tìm ra nghiệm có chi phí nhỏ nhất không?
- Các tham số để tính độ phức tạp về không gian và thời gian:
  - *b:* số trạng thái kề (maximum branching factor of the search tree)
  - *d:* độ sâu của trạng thái kết thúc (depth of the least-cost solution)

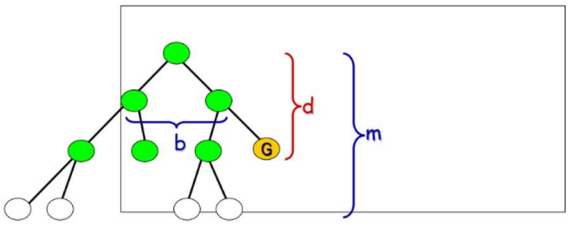
17

Breadth first search

1. Khởi tạo hàng đợi Q chỉ chứa trạng thái ban đầu
2. **Loop do**
   - 2.1 **If** (Q rỗng) **then**
     {thông báo tìm kiếm thất bại; kết thúc}
   - 2.2 Loại trạng thái u ở đầu hàng đợi Q;
   - 2.3 **If** (u là trạng thái kết thúc) **then**
     {thông báo tìm kiếm thành công; kết thúc}
   - 2.4 **For** (mỗi trạng thái v kề u) **do**
     {Đặt v vào cuối hàng đợi Q;}

Đánh giá BFS

- If a goal node is found on depth d of the tree, all nodes up till that depth are created.



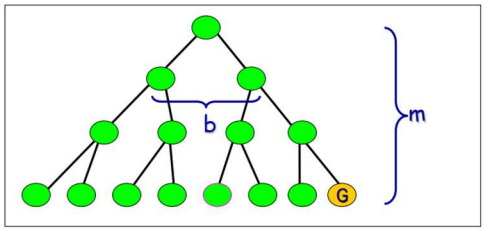
- Completeness: Yes, if *b* is finite
- Time complexity: $1+b+b^2+\ldots+b^d = O(b^d)$
- Space complexity: $O(b^d)$
- Optimality: Yes (1 per step)

Depth first search

1. Khởi tạo ngăn xếp S chỉ chứa trạng thái ban đầu
2. **Loop do**
   - 2.1 **If** (S rỗng) **then**
     {thông báo tìm kiếm thất bại; kết thúc}
   - 2.2 Loại trạng thái u ở đầu ngăn xếp S;
   - 2.3 **If** (u là trạng thái kết thúc) **then**
     {thông báo tìm kiếm thành công; kết thúc}
   - 2.4 **For** (mỗi trạng thái v kề u) **do**
     {Đặt v vào đầu ngăn xếp S;}

Đánh giá DFS

- In the worst case: the (only) goal node may be on the right-most branch,



- Completeness: No
- Time complexity: $O(b^d)$
- Space complexity: $O(m*b)$
- Optimality: No

5.4. Tìm kiếm với độ sâu giới hạn

Procedure Depth_Limited_Search(d);
begin
1. Khởi tạo danh sách L chỉ chứa trạng thái ban đầu u_0;
   depth(u_0) ← 0;
2. **loop do**
   2.1. **if L** rỗng **then**
      {thông báo thất bại; stop**};**
   2.2. Loại trạng thái u ở đầu danh sách L;
   2.3. **if u** là trạng thái kết thúc **then**
      {thông báo thành công; stop**};**
   2.4. **if** depth(u) <= d **then**
      for mỗi trạng thái **v** kề **u do**
         {Đặt **v** vào đầu danh sách L;
         depth(v) ← depth(u) + 1**};**
end;

Combines the best of breadth first and depth first search strategies.

Function Iterative-deepening-Search(*problem*) **returns** a solution, or failure
  for *depth* = 0 **to** ∞ **do**
    result ← Depth-Limited-Search(*problem*, *depth*)
    if *result* succeeds **then return** *result*
  end
  return failure

- Completeness: Yes,
- Time complexity: $O(b^d)$
- Space complexity: $O(bd)$
- Optimality: Yes, if step cost = 1

* Uniform-Cost Search: Tính d[i] là khoảng cách từ đỉnh i đến đỉnh gốc. Duyệt theo priority queue, ưu tiên theo d[i]

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening
Complete?	Yes	Yes	No	No	Yes
Time	$O(b^{d+1})$	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^l)$	$O(b^d)$
Space	$O(b^{d+1})$	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(bl)$	$O(bd)$
Optimal?	Yes	Yes	No	No	Yes

* Best-first Search = BFS + hàm đánh giá

Lược đồ Tìm kiếm tốt nhất đầu tiên

1. Khởi tạo danh sách **L** chỉ chứa trạng thái ban đầu
2. **Loop do**
   - 2.1 **If** (L rỗng) **then**
     {thông báo tìm kiếm thất bại; kết thúc}
   - 2.2 Loại trạng thái u ở đầu danh sách **L**;
   - 2.3 **If** (u là trạng thái kết thúc) **then**
     {thông báo tìm kiếm thành công; kết thúc}
   - 2.4 **For** (mỗi trạng thái v kề u) **do**
     {Đặt v vào danh sách **L** sao cho **L** được sắp xếp thep thứ tự tốt đến xấu của hàm đánh giá;}

Đặc trưng của tìm kiếm tốt nhất đầu tiên

- Complete? **No** – Có thể bị quẩn, e.g., Iasi → Neamt → Iasi → Neamt →
  Complete in finite space with repeated-state checking
- Time? $O(b^m)$, m là độ sâu
- Space? $O(b^m)$
- Optimal? **No**

Một hàm đánh giá tốt có thể giảm thời gian và không gian nhớ một cách đánh kể

Chiến lược Tìm kiếm Beam
Beam Search

- Tương tự như tìm kiếm tốt nhất đầu tiên
- Tuy nhiên chỉ phát triển k đỉnh ở mức tiếp theo chứ không phát triển toàn bộ
- Ưu điểm: độ phức tạp tính toán tốt hơn
- Nhược điểm: không tìm kiếm toàn bộ, nên có thể không tìm thấy đỉnh tối ưu nhất

* Hill-climbing Search = DFS + hàm đánh giá

Thuật toán tìm kiếm leo đồi

1. Khởi tạo danh sách **L** chỉ chứa trạng thái ban đầu;
2. **Loop do**
   - 2.1. **if** L rỗng then
     { thông báo thất bại; **stop**};
   - 2.2. Loại trạng thái u ở đầu danh sách **L**;
   - 2.3. **if** u là trạng thái kết thúc **then**
     {thông báo thành công, **stop**};
   - 2.4. **for** mỗi trạng thái v kề u do đặt v vào **L1**;
   - 2.5. Sắp xếp **L1** theo thứ tự tăng dần của hàm đánh giá sao cho trạng thái tốt nhất ở đầu danh sách **L1**
   - 2.6. Chuyển danh sách **L1** vào đầu danh sách **L** sao cho đầu danh sách **L1** trở thành đầu **L**

Tìm kiếm A*

- A* sử dụng hàm $f(u) = g(u) + h(u)$
  - $g(u)$: tổng chi phí từ xuất phát đến u
  - $h(u)$: ước lượng chi phí từ u đến đích.
  - $f(u)$: ước lượng tổng chi phí của phương án khi qua u.
- Tìm kiếm tốt nhất đầu tiên → sử dụng $h(u)$

  ➔ Hiệu quả nhưng không tối ưu
- Tìm kiếm theo chiều rộng → sử dụng $g(u)$

  ➔Tối ưu nhưng không hiệu quả

Lược đồ thuật toán A*

1. Khởi tạo danh sách L chỉ chứa trạng thái ban đầu
2. **Loop do**
   - 2.1 **If** (L rỗng) **then**
     {thông báo tìm kiếm thất bại; kết thúc}
   - 2.2 Loại trạng thái u ở đầu danh sách L;
   - 2.3 **If** (u là trạng thái kết thúc) **then**
     {thông báo tìm kiếm thành công; kết thúc}
   - 2.4 **For** (mỗi trạng thái v kề u) **do**
     {g(v) := g(u) + k(u,v);
     f(v) := g(v) + h(v);
     Đặt v vào danh sách L sao cho L được sắp xếp thep thứ tự tốt đến xấu của hàm đánh giá f;}

Tìm kiếm A*

- Tìm kiếm A* sử dụng hàm $f(u) = g(u) + h(u)$ nhỏ nhất.
- Câu hỏi: A* có tối ưu (optimal) và bảo đảm tìm được nghiệm không (complete)?
- Yes! nếu h(n) không vượt quá giá trị thực tế từ trạng thái n tới đích.

Tìm kiếm A* trọng số

- **Ý tưởng:** Chọn một hàm chấp nhận được, nhân với $\alpha > 1$ và thực hiện tìm kiếm A*.

➔ Sẽ ít trạng thái được phát triển nhưng kết quả có thể tối ưu cục bộ (chênh lệch với kết quả tối ưu không quá α lần)

Cải tiến thuật toán A* bằng nhánh cận

1. Khởi tạo danh sách L chỉ chứa trạng thái ban đầu
2. **Loop do**
   - 2.1 **If** (L rỗng) **then**
     {thông báo tìm kiếm thất bại; kết thúc}
   - 2.2 Loại trạng thái u ở đầu danh sách L;
   - **2.3' If L bao gồm đường đi P kết thúc tại trạng thái X với giá là cost_P, và đường đi Q bao gồm X với giá là cost_Q và cost_P >= cost_Q then**
     { Xóa P khỏi L}
   - 2.3 **If** (u là trạng thái kết thúc) **then**
     {thông báo tìm kiếm thành công; kết thúc}
   - 2.4 **For** (mỗi trạng thái v kề u) **do**
     {g(v) := g(u) + k(u,v);
     f(v) := g(v) + h(v);
     Đặt v vào danh sách L sao cho L được sắp xếp thep thứ tự tốt đến xấu của hàm đánh giá f;}

Các hàm trong chiến lược Minimax

Hàm gán giá trị max:

```
Function MaxValue(u);
Begin
    If u là lá then MaxValue(u) ← f(u)
    Else MaxValue(u) ← max {MinValue(v) | v là các đỉnh con của u}
End;
```

Hàm gán giá trị min:

```
Function MinValue(u);
Begin
    If u là lá then MinValue(u) ← f(u)
    Else MinValue(u) ← min {MaxValue(v) | v là các đỉnh con của u}
End;
```

Thủ tục minimax:

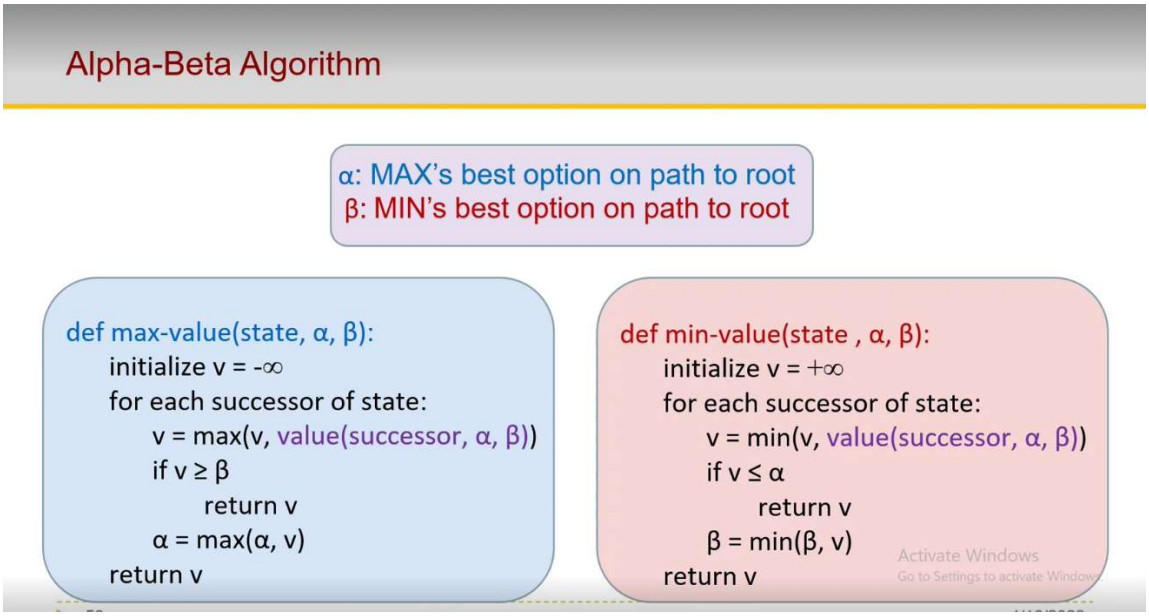
```
Procedure Minimax(u,v);
Begin
    Value ← - ∞;
    For mỗi w là đỉnh con của u do
        If Value <= MinValue(w) then
        Begin Value ← MinValue(w);
              v ← w; end;
End;
```

Đánh giá về chiến lược Minimax:

1. **Tính đủ**
   - Có (nếu cây tìm kiếm là hữu hạn).
2. **Tính tối ưu**
   - có (đối với đối thủ luôn ra nước tối ưu)
3. **Độ phức tạp thời gian?**
   - $O(b^m)$ (với m là độ cao của cây, tại mỗi đỉnh có b nước đi)
4. **Độ phức tạp không gian?**
   - O(bm) (DFS)

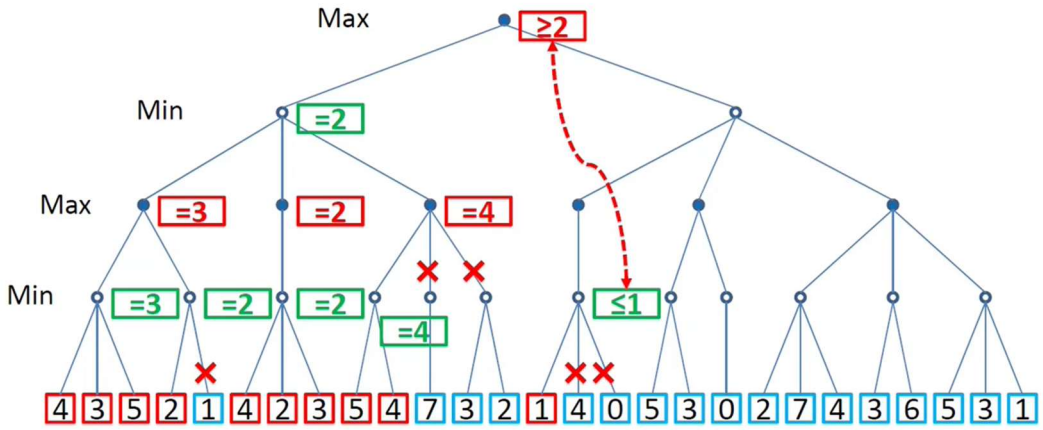
- Ví dụ cờ vua, thông thường b ≈ 35, m ≈100 → Tìm lời giải chính xác và tối ưu là không thể được

* $\alpha - \beta$ Pruning: cắt bỏ khi Tổ tiên của β (max) $\geq \alpha$(min)



MiniMax with $\alpha\beta$-pruning

- **"Deep" cut-off: Ancestor β-node ≥ α-node**



Knowledge and Reasoning

- **Phép tuyển: ∨ ~ hoặc**
- **Phép hội: ∧ ~ và**

Các luật tương đương quan trọng

- **Luật đồng nhất (Identity)**
  - p ∧ T <=> p
  - p ∨ F <=> p
- **Luật trội (Domination)**
  - p ∨ T <=> T
  - p ∧ F <=> F
- **Luật luỹ đẳng (Idempotent)**
  - p ∨ p <=> p
  - p ∧ p <=> p

Các luật tương đương quan trọng

- **Luật phủ định kép (Double negation)**
  - ¬(¬p) <=> p
- **Luật giao hoán (Commutative)**
  - p ∨ q <=> q ∨ p
  - p ∧ q <=> q ∧ p
- **Luật kết hợp (Associative)**
  - (p ∨ q) ∨ r <=> p ∨ (q ∨ r)
  - (p ∧ q) ∧ r <=> p ∧ (q ∧ r)

Các luật tương đương quan trọng

- **Luật phân phối (Distributive)**
  - p ∨ (q ∧ r) <=> (p ∨ q) ∧ (p ∨ r)
  - p ∧ (q ∨ r) <=> (p ∧ q) ∨ (p ∧ r)
- **Luật De Morgan**
  - ¬(p ∨ q) <=> ¬p ∧ ¬q
  - ¬(p ∧ q) <=> ¬p ∨ ¬q
- **Các luật hữu ích khác**
  - p ∨ ¬p <=> T
  - p ∧ ¬p <=> F
  - p → q <=> (¬p ∨ q)

Thuật toán nhận biết hằng đúng

Input: Biểu thức A

Output:Dạng chuẩn tắc hội, A là hằng đúng?

Thuật toán

Bước 1: Khử phép toán kéo theo (→) trong A bằng cách áp dụng X→Y ↔ ¬X∨Y

Bước 2: Đưa phép toán phủ định về trực tiếp liên quan đến từng mệnh đề sơ cấp bằng cách áp dụng ¬(A ∨ B) ↔ (¬A ∧¬B) hoặc ¬(A ∧ B) ↔ (¬A ∨¬B)

Bước 3: Đưa về dạng chuẩn tắc hội bằng cách áp dụng X∨(Y∧Z) ↔ (X∨Y)∧(X∨Z)

Bước 4: Kiểm tra điều kiện và kết luận

Nếu mỗi tuyển sơ cấp đều chứa mệnh đề sơ cấp và phủ định của nó → Hằng đúng

Thuật toán nhận biết hằng sai

Input: Biểu thức A

Output:Dạng chuẩn tắc tuyển của A, A là hằng sai?

Thuật toán

Bước 1: Khử phép toán kéo theo (→) trong A bằng cách áp dụng X→Y ↔ ¬X∨Y

Bước 2: Đưa phép toán phủ định về trực tiếp liên quan đến từng mệnh đề sơ cấp bằng cách áp dụng ¬(A ∨ B) ↔ (¬A ∧¬B) hoặc ¬(A ∧ B) ↔ (¬A ∨¬B)

Bước 3: Đưa về dạng chuẩn tắc tuyển bằng cách áp dụng X∧(Y∨Z) ↔ (X∧Y)∨(X∧Z)

Bước 4: Kiểm tra điều kiện và kết luận

Nếu mỗi hội sơ cấp đều chứa mệnh đề sơ cấp và phủ định của nó → Hằng sai

Các quy tắc suy diễn

- **(1)** Quy tắc suy diễn rút gọn (**Simplification**)

  $(p \wedge q) \rightarrow p$

  $\dfrac{p}{\therefore\ p}$ $\dfrac{q}{}$

  - Ví dụ: Hà Nội là thủ đô Việt Nam và trời đang mưa. Do đó trời đang mưa.

- **(2)** Quy tắc suy diễn cộng (**Addition**)

  $p \rightarrow (p \vee q)$

  $\dfrac{p}{\therefore\ p \vee q}$

  - Ví dụ: Hà Nội là thủ đô Việt Nam. Do đó, Hà Nội là thủ đô Việt Nam hoặc trời đang mưa.

Các quy tắc suy diễn

- **(3)** Quy tắc suy diễn khẳng định (**Modus Ponens**)

  $(p \wedge (p \rightarrow q)) \rightarrow q$

  $\dfrac{p \qquad p \rightarrow q}{\therefore\ q}$

  - Ví dụ: Bình được thưởng cuối năm thì Bình đi chơi Đà Lạt. Bình được thưởng cuối năm. Do đó, Bình đi chơi Đà Lạt.

- **(4)** Quy tắc suy diễn phủ định (**Modus Tollens**)

  $[\neg q \wedge (p \rightarrow q)] \rightarrow \neg p$

  $\dfrac{\neg q \qquad p \rightarrow q}{\therefore\ \neg p}$

  - Ví dụ: Bình được thưởng cuối năm thì Bình đi chơi Đà Lạt. Bình không đi chơi Đà Lạt. Do đó, Bình không được thưởng cuối năm .

Các quy tắc suy diễn

- **(5)** Quy tắc suy diễn tam đoạn luận (**Hypothetical Syllogism**)

$$[(p \to q) \wedge (q \to r)] \to (p \to r) \qquad \frac{p \to q,\ q \to r}{\therefore\ p \to r}$$

  - Ví dụ: Bình được thưởng cuối năm thì Bình đi chơi Đà Lạt. Bình đi chơi Đà Lạt thì Bình sẽ thăm Thiền Viện. Do đó, Bình được thưởng cuối năm thì Bình sẽ thăm Thiền Viện.
- **(6)** Quy tắc suy diễn tam đoạn luận rời (**Disjunctive Syllogism**)

$$[(p \vee q) \wedge \neg p] \to q \qquad \frac{p \vee q,\ \neg p}{\therefore\ q}$$

  - Ví dụ: An dậy sớm hoặc An đi làm muộn. An không dậy sớm. Do đó, An đi làm muộn.

Các quy tắc suy diễn

- **(7)** Quy tắc suy diễn mâu thuẫn

$$(p1 \wedge p2) \to q \equiv (p1 \wedge p2 \wedge \neg q) \to F$$

  - Ví dụ: ?
- **(8)** Quy tắc suy diễn theo trường hợp

$$(p \to r) \wedge (q \to r) \to ((p \vee q) \to r) \qquad \frac{p \to r,\ q \to r}{\therefore (p \vee q) \to r}$$

Các luật suy diễn

- Luật suy diễn khẳng định
- Luật suy diễn phủ định
- Luật bắc cầu
- Luật loại bỏ hội
- Luật đưa vào hội
- Luật đưa vào tuyển
- Luật phân giải

$$\frac{\alpha \Rightarrow \beta, \alpha}{\beta} \qquad \frac{\alpha \Rightarrow \beta, \neg\beta}{\neg\alpha} \qquad \frac{\alpha \Rightarrow \beta, \beta \Rightarrow \gamma}{\alpha \Rightarrow \gamma}$$

$$\frac{\alpha 1 \wedge \ldots \wedge \alpha i \wedge \ldots \alpha m}{\alpha i} \qquad \frac{\alpha 1, \ldots, \alpha i, \ldots, \alpha m}{\alpha 1 \wedge \ldots \wedge \alpha i \wedge \ldots \wedge \ldots \alpha m}$$

$$\frac{\alpha i}{\alpha 1 \vee \ldots \vee \alpha i \vee \ldots \vee \alpha m} \qquad \frac{\alpha \vee \beta, 7\beta \vee \gamma}{\alpha \vee \gamma}$$

Luật phân giải (Resolution)

- Luật phân giải là sự tổng quát của các luật khẳng định, phụ định và luật bắc cầu.
- Luật phân giải trên các câu tuyển

$$\frac{A_1 \vee A_2 \vee \ldots \vee A_m \vee C,\ \neg C \vee B_1 \vee B_2 \vee \ldots \vee B_n}{A_1 \vee \ldots \vee A_m \vee B_1 \vee \ldots \vee B_n}$$

Chứng minh bác bỏ bằng luật phân giải

- Ý tưởng:

Từ G muốn chứng minh A, ta chứng minh từ G và ⌉A sẽ suy ra được câu rỗng (hay mâu thuẫn)

Thuật toán phân giải

Tư tưởng

- Hoạt động dựa trên phương pháp chứng minh phản chứng và luật phân giải.
- Lần lượt ghép hai cặp câu với nhau và áp dụng định luật phân giải sinh ra câu mới đưa vào tập tri thức.
- Cứ làm như vậy cho đến khi không sinh ra được câu mới thì thôi.

Thuật toán phân giải
Đặc tả

Procedure Robinson;
Input : tập G các công thức, công thức cần chứng minh H.
Begin
1. Thêm ⌉H vào G;
2. Đưa G về tập các câu tuyển theo dạng chuẩn tắc hội.
2. **Repeat**
2.1 Chọn hai câu A, B thuộc G.
2.2 **If** (A, B phân giải được) **then**
Thêm câu mới Res(A, B) vào G.
Until (Nhận được câu rỗng **or** Không sinh ra được câu mới);
3. **If** (nhận được câu rỗng) **then** Chứng minh được H;
Else Không chứng minh được H;

End;

THUẬT GIẢI ROBINSON

1. Biến đổi tất cả các câu thành dạng CNF
2. Lấy phủ định kết luận, đưa vào KB
3. Lặp
   a. Nếu trong KB có chứa hai mệnh đề phủ định nhau thì trả về ***false //trả về false***
   => vô lý
   b. Nếu có hai mệnh đề chứa các literal phủ định nhau thì áp dụng hợp giải.
   c. Lặp cho đến khi không thể áp dụng tiếp luật hợp giải.
4. Trả về ***true***

Dịch với lượng từ

Giả sử có 02 vị từ S(x) và P(x)

- **Phát biểu "với mọi" sẽ ngầm chỉ phép "kéo theo"**
  - *Tất cả S(x) đều P(x)*
    - ∀x (S(x) → P(x))
  - *Không S(x) nào là P(x)*
    - ∀x(S(x) → ¬P(x))
- **Phát biểu "tồn tại" sẽ ngầm chỉ phép "hội"**
  - *Một số S(x) là P(x)*
    - ∃x (S(x) ∧ P(x))
  - *Một vài S(x) không là P(x)*
    - ∃x (S(x) ∧ ¬P(x))

Lập luận trong Logic vị từ cấp I
Ví dụ

Cho: An là con trai; Bình là con gái; Tóc của con gái dài hơn tóc của con trai;
Chứng minh: Tóc của Bình dài hơn tóc của An

KB: (1) Con_trai(An); (2) Con_gái(Bình);
(3) ∀x, ∀y Con_trai(x) ^ Con_gái(y) → Tóc_dài_hơn(y,x)
Cần chứng minh: Tóc_dài_hơn(Bình,An)

Từ (1), (2), đưa vào luật ^ : Con_trai(An) ^ Con_gái(Bình)
Từ (3), loại bỏ ∀ với {x/An, y/Bình} : Con_trai(An) ^ Con_gái(Bình)

Công thức Bayes

» Nếu B_1, B_2,..., B_n là một hệ đầy đủ các biến cố và A là 1 biến cố với P(A) > 0 thì với mỗi k=1,2,...,n

$$P(B_k|A) = \frac{P(B_k)P(A|B_k)}{\sum_{i=1}^{n} P(B_i)P(A|B_i)} = \frac{P(B_k)P(A|B_k)}{P(A)} = \frac{P(AB_k)}{P(A)}$$

- Các xác suất $P(B_1)$,...,$P(B_n)$ được gọi là các *xác suất tiên nghiệm* (trước thí nghiệm)
- Các xác suất $P(B_1|A)$,...,$P(B_n|A)$ được gọi là các *xác suất hậu nghiệm* (sau thí nghiệm)

$$P(A) = \sum_{i=1}^{n} P(A \mid Bi) * P(Bi)$$

Sự độc lập

- Biến ngẫu nhiên X độc lập với biến ngẫu nhiên Y nếu:

$$\Pr(X|Y) = \Pr(X)$$

- Biến ngẫu nhiên X độc lập với biến ngẫu nhiên Y có điều kiện Z:

$$\Pr(X|Y,Z) = \Pr(X|Z)$$

Machine Learning

Học máy và AI

- Giải quyết vấn đề bằng suy diễn với thông tin được cung cấp trước → không cải thiện được hiệu quả trong tương lai dựa trên kinh nghiệm thu được
- Cải tiến hiệu quả giải quyết bài toán thông qua quan sát và sự dạy bảo
- Học máy như là một thành phần quan trọng của AI

Biểu diễn một bài toán học máy [Mitchell, 1997]

- **Một công việc (nhiệm vụ) T**
- **Đối với các tiêu chí đánh giá hiệu năng P**
- **Thông qua (sử dụng) kinh nghiệm E**

Nhận dạng mặt người

- ***T: Nhận dạng mặt người trên các bức ảnh đã cho trước***
- ***P: Tỷ lệ (%) độ chính xác mặt người nhận dạng được các bức ảnh***
- ***E: Một tập các khuôn mặt người đã được xác định trước.***

Phân lớp Bayes

$$p(h \mid d) = \frac{P(d \mid h)P(h)}{P(d)} \qquad h_{MAP} = \arg\max_{h \in H} P(h \mid d)$$

$$h_{MAP} = \arg\max_{h \in H} P(h_j | a_1, a_2, \dots, a_n)$$

$$= \arg\max_{h \in H} \frac{P(a_1, a_2, \dots, a_n | h_j) P(h_j)}{P(a_1, a_2, \dots, a_n)}$$

$$h_{MAP} = \arg\max_{h \in H} P(a_1, a_2, \dots, a_n | h_j) P(h_j)$$

Giả sử Bayes ngây thơ: Các thuộc tính mà mô tả dữ liệu là độc lập điều kiện theo bởi giả thuyết phân lớp:

$$P(\mathbf{d} \mid h) = P(a_1, \dots, a_T \mid h) = \prod_t P(a_t \mid h)$$

- Đây là giả sử ngây thơ (hồn nhiên), vì vậy thường trái với thực tế.
- Mặc dù vậy, nó làm việc tốt trong các ứng dụng thực tế

Support Vector Machines (1)
(Máy vector hỗ trợ)

- Máy vector hỗ trợ được đề xuất bởi V.Vapnik và đồng nghiệp vào những năm 1970 ở Nga và phổ biến vào những năm 1990
- SVM là phương pháp phân lớp tuyến tính bằng cách xác định một siêu phẳng (hyperplane) để phân tách hai lớp của dữ liệu. Ví dụ, lớp có nhãn âm (negative) và lớp có nhãn dương (positive)
- Các hàm nhân (kernel fuctionals), cũng được gọi là hàm biến đổi (transformation functions), được dùng cho trường hợp phân lớp phi tuyến
- SVM có nền tảng lí thuyết chặt chẽ dựa trên nhiều định lí toán học
- SVM tốt và phù hợp đối với những bài toán phân lớp có không gian biểu diễn thuộc tính lớn (nhiều thuộc tính)
- SVM là một trong các phương pháp tốt cho bài toán phân lớp văn bản

Support Vector Machines (2)
(Máy vector hỗ trợ)

- Biểu diễn tập r các ví dụ huấn luyện (tranning examples)

$$\{(\boldsymbol{x_1}, y_1), (\boldsymbol{x_2}, y_2), \dots, (\boldsymbol{x_r}, y_r)\}$$

  - x_i là một vector đầu vào được biểu diễn trong không gian $X \subseteq R^n$
  - y_i là nhãn lớp (giá trị đầu ra), $y_i \in \{-1, +1\}$

- Đối với một ví dụ x_i:

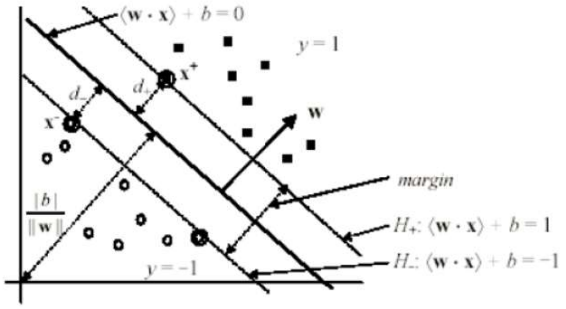
$$y_i = \begin{cases} +1 \; nếu < w \cdot x_i > +b \geq 0 \\ -1 \; nếu < w \cdot x_i > +b < 0 \end{cases}$$

- SVM xác định một hàm phân tách tuyến tính

$$f(x) =< w \cdot x > +b$$

Trong đó w là vector trọng số các thuộc tính; b là giá trị thực

- SVM lựa chọn mặt siêu phẳng phân tách có lề (margin) cực đại
- Lý thuyết học máy đã chỉ ra rằng một mặt siêu phẳng phân tách như thế sẽ tối thiểu hóa giới hạn lỗi (phân lớp) mắc phải.



32

SVM - Dữ liệu phân tách được tuyến tính

- Giả sử tập dữ liệu (tập các dữ liệu huấn luyện) có thể phân tách được một cách tuyến tính
- Xét một ví dụ các lớp dương $(x^+, 1)$ và ví dụ các lớp âm $(x^-, -1)$ gần nhất đối với siêu phẳng phân tách H_0 $(< w \cdot x > +b = 0)$

H_+ đi qua x^+ và song song với H_0

H_- đi qua x^- và song song với H_0

$$< w \cdot x^+ > +b = \quad 1$$
$$< w \cdot x^- > +b = -1$$
$$< w \cdot x_i > +b \geq \quad 1, nếu\ y_i = +1$$
$$< w \cdot x_i > +b \leq -1, nếu\ y_i = -1$$

- Mức lề là khoảng cách giữa hai siêu phẳng lề H_+ và H_-
  - d_+ là khoảng cách giữa H_+ và H_0
  - d_- là khoảng cách giữa H_- và H_0
- Khoảng cách (trực giao) từ một điểm x_i đến mặt siêu phẳng $(< w \cdot x > +b = 0$ là $\frac{|<w.x_i>+b|}{||w||}$, trong đó $||w||$ là độ dài của w

$||w|| = \sqrt{< w \cdot w >} = \sqrt{w_1^2 + \cdots + w_n^2}$

Tính toán mức lề

$$d_+ = \frac{|\langle \mathbf{w} \cdot \mathbf{x}^+ \rangle + b|}{\|\mathbf{w}\|} = \frac{|1|}{\|\mathbf{w}\|} = \frac{1}{\|\mathbf{w}\|} \qquad d_- = \frac{|\langle \mathbf{w} \cdot \mathbf{x}^- \rangle + b|}{\|\mathbf{w}\|} = \frac{|-1|}{\|\mathbf{w}\|} = \frac{1}{\|\mathbf{w}\|}$$

$$margin = d_+ + d_- = \frac{2}{\|\mathbf{w}\|}$$

- SVM học là một phân lớp nhằm cực đại hóa mức lề
- Giải quyết bài toán tối ưu bậc 2: Tìm w và b sao cho $margin = \frac{2}{||w||}$ cực đại với điều kiện $\begin{cases} < w \cdot x_i > +b \geq \quad 1, nếu\ y_i = +1 \\ < w \cdot x_i > +b \leq -1, nếu\ y_i = -1 \end{cases}$
- Giải quyết bài toán này vẫn là một nhiệm vụ khó khăn

Nới lỏng các điều kiện

- Để làm việc với các dữ liệu chứa nhiễu, cần nới lỏng các điều kiện lề bằng cách sử dụng các biến slack $\xi_i (\geq 0)$

$$\begin{cases} < w \cdot x_i > +b \geq \quad 1 - \xi_i, nếu\ y_i = +1 \\ < w \cdot x_i > +b \leq -1 + \xi_i, nếu\ y_i = -1 \end{cases}$$

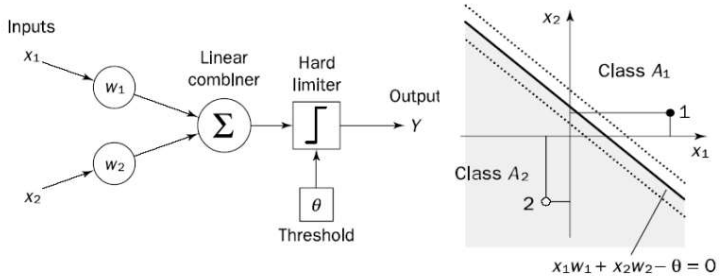
- Đối với một ví dụ nhiễu (lỗi): $\xi_i > 1$

- $(\sum \xi_i)$ là giới hạn trên của lỗi của ví dụ huấn luyện

Perceptron là mạng Neuron với 0 lớp ẩn nên có nhiều hạn chế (vd: không học được XOR)

Perceptron

- Là mạng neuron nhân tạo đơn giản nhất gồm một neuron với các trọng số **có thể điều chỉnh được** và một **giới hạn cứng**.



- Perceptron phân loại các input thành hai lớp A_1 và A_2 bị chia cách bởi một siêu phẳng cho bởi hàm phân biệt tuyến tính

$$\sum_{i=1}^{n} x_i w_i - \theta = 0$$

Trang 42

Perceptron

- Một perceptron học bằng cách điều chỉnh **siêu phẳng phân biệt** theo sai số $e(p) = Y_d(p) - Y(p)$ với $Y_d(p)/Y(p)$ là kết xuất thực/mong muốn tại bước p

- Nếu e(p) > 0 → tăng *Y(p),* ngược lại giảm *Y(p).*
- Nếu $x_i(p) > 0$, tăng $w_i(p)$ sẽ làm tăng Y(p). Ngược lại nếu $x_i(p) < 0$, tăng $w_i(p)$ sẽ làm giảm Y(p).

- Do đó, **luật học perceptron** có thể được viết là

$$w_i(p+1) = w_i(p) + \alpha \times x_i(p) \times e(p)$$

với α là **tốc độ học**

Perceptron – Thuật toán Học

- **Khởi tạo**: $(w_1, w_2, \ldots, w_n, t) \in [-0.5, 0.5]$

- **Kích hoạt**: tính kết xuất thực sự tại lần lặp p = 1

$$Y(p) = step\left[\sum_{i=1}^{n} x_i(p) w_i(p) - \theta\right]$$

- **Cập nhật trọng số:**

$$w_i(p+1) = w_i(p) + \Delta w_i(p)$$

trong đó $\Delta w_i(p) = \alpha \times x_i(p) \times e(p)$ **(luật học delta)**

- **Lặp:** tăng p lên 1 và lặp lại bước 2.

Perceptron – Ví dụ

- Học hàm AND

x_1	x_2	$x_1 \cap x_2$	$x_1 \cup x_2$	$x_1 \oplus x_2$
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

- Khởi tạo trọng số: $w_1 = 0.3$, $w_2 = -0.1$, $\theta = 0.2$, tốc độ học $\alpha = 0.1$
- Lặp:

Thế hệ	Input		Kết xuất mong muốn	Trọng số ban đầu		Kết xuất thực	Sai số	Trọng số cuối	
	x_1	x_2	Y_d	w_1	w_2	Y	e	w_1	w_2
1	0	0	0	0.3	-0.1	0	0	0.3	-0.1
	0	1	0	0.3	-0.1	0	0	0.3	-0.1
	1	0	0	0.3	-0.1	1	-1	0.2	-0.1
	1	1	1	0.2	-0.1	0	1	0.3	0.0

Perceptron – Ví dụ (tt)

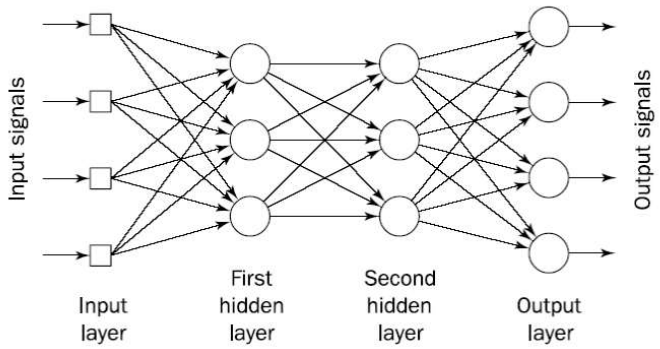
Thế hệ	Inputs		Kết xuất mong muốn	Trọng số ban đầu		Kết xuất thực	Sai số	Trọng số cuối	
	x_1	x_2	Y_d	w_1	w_2	Y	e	w_1	w_2
2	0	0	0	0.3	0.0	0	0	0.3	0.0
	0	1	0	0.3	0.0	0	0	0.3	0.0
	1	0	0	0.3	0.0	1	-1	0.2	0.0
	1	1	1	0.2	0.0	0	0	0.2	0.0
3	0	0	0	0.2	0.0	0	0	0.2	0.0
	0	1	0	0.2	0.0	0	0	0.2	0.0
	1	0	0	0.2	0.0	1	-1	0.1	0.0
	1	1	1	0.1	0.0	0	1	0.2	0.1

Perceptron – Ví dụ (tt)

Thế hệ	Inputs		Kết xuất mong muốn	Trọng số ban đầu		Kết xuất thực	Sai số	Trọng số cuối	
	x_1	x_2	Y_d	w_1	w_2	Y	e	w_1	w_2
4	0	0	0	0.2	0.1	0	0	0.2	0.1
	0	1	0	0.2	0.1	0	0	0.2	0.1
	1	0	0	0.2	0.1	1	-1	0.1	0.1
	1	1	1	0.1	0.1	1	0	0.1	0.1
5	0	0	0	0.1	0.1	0	0	0.1	0.1
	0	1	0	0.1	0.1	0	0	0.1	0.1
	1	0	0	0.1	0.1	0	0	0.1	0.1
	1	1	1	0.1	0.1	1	0	0.1	0.1

Mạng Neuron nhiều lớp

- Một mạng neuron lan truyền tiến gồm một **lớp nhập**, ít nhất một **lớp ẩn**, và một **lớp xuất**.



Mạng Neuron nhiều lớp

- Lớp nhập: nhận các input và phân phối chúng cho tất cả neuron trong lớp ẩn
- Lớp xuất: biểu diễn kết quả của toàn mạng
- Lớp ẩn:
  - Dò tìm các đặc trưng.
  - Các neuron trong lớp này "ẩn" các kết xuất mong muốn của chúng.
  - Mạng một lớp ẩn có thể biểu diễn bất kỳ hàm liên tục nào.
  - Mạng hai lớp ẩn có thể biểu diễn các hàm không liên tục.

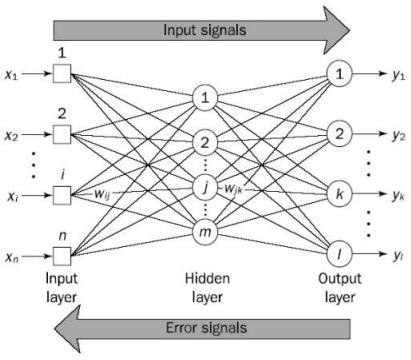
Học trong mạng neuron nhiều lớp

- Quá trình học gồm hai pha:
  - Lan truyền các mẫu input từ lớp nhập đến lớp xuất, tại mỗi neuron tính

$$X = \sum_{i=1}^{n} x_i w_i - \theta$$

$$Y^{sigmoid} = \frac{1}{1 + e^{-X}}$$

  - Lan truyền ngược sai số từ lớp xuất và cập nhật các trọng số



Luật học trong mạng neuron nhiều lớp

- Tại nút xuất, tính sai số

$$e_k(p) = y_{d,k}(p) - y_k(p)$$

với là kết xuất mong muốn ở nút k tại vòng lặp thứ p

- Cập nhật sai số tương tự luật học perceptron

$$w_{jk}(p+1) = w_{jk}(p) + \Delta w_{jk}(p)$$

với

$$\Delta w_{jk}(p) = \alpha \times y_j(p) \times \delta_k(p)$$

gradient sai số tại neuron *k* của lớp xuất

Gradient sai số

- Gradient sai số = đạo hàm hàm kích hoạt nhân với sai số tại neuron xuất

$$\delta_k(p) = \frac{\partial y_k(p)}{\partial X_k(p)} \times e_k(p)$$

tính đạo hàm ta được

$$\delta_k(p) = y_k(p) \times [1 - y_k(p)] \times e_k(p)$$

trong đó

$$y_k(p) = \frac{1}{1 + \exp[-X_k(p)]}$$

Luật học trong mạng neuron nhiều lớp (tt)

- Tại nút ẩn, ta có thể áp dụng cùng công thức với lớp xuất:

$$\Delta w_{ij}(p) = \alpha \times x_i(p) \times \delta_j(p)$$

trong đó $\delta_j(p)$ là gradient sai số tại neuron *j* trong lớp ẩn

$$\delta_j(p) = y_j(p) \times [1 - y_j(p)] \times \sum_{k=1}^{l} \delta_k(p) w_{jk}(p)$$

với l là số neuron của nút xuất và

$$y_j(p) = \frac{1}{1 + e^{-X_j(p)}}$$

$$X_j(p) = \sum_{i=1}^{n} x_i(p) \times w_{ij}(p) - \theta_j$$

Thuật toán lan truyền ngược

- **Bước 1**: ***Khởi tạo***

  Đặt giá trị ngẫu nhiên các trọng số và ngưỡng của mạng

- **Bước 2*: Kích hoạt***

  a) Tính kết xuất thực sự của các neuron trong lớp ẩn:

$$y_j(p) = sigmoid\left[\sum_{i=1}^{n} x_i(p) \times w_{ij}(p) - \theta_j\right]$$

  b) Tính kết xuất thực sự của các neuron trong lớp xuất:

$$y_k(p) = sigmoid\left[\sum_{j=1}^{m} x_{jk}(p) \times w_{jk}(p) - \theta_k\right]$$

Thuật toán lan truyền ngược (tt)

- **Bước 3**: ***Huấn luyện trọng số***

  a) Tính gradient sai số cho các neuron lớp xuất:

$$\delta_k(p) = y_k(p) \times [1 - y_k(p)] \times e_k(p)$$
$$\Delta w_{jk}(p) = \alpha \times y_j(p) \times \delta_k(p)$$

  Cập nhật các trọng số của neuron lớp xuất:

$$w_{jk}(p+1) = w_{jk}(p) + \Delta w_{jk}(p)$$

  b) Tính gradien sai số và cập nhật trọng số lớp ẩn

$$\delta_j(p) = y_j(p) \times [1 - y_j(p)] \times \sum_{k=1}^{l} \delta_k(p) \times w_{jk}(p)$$
$$\Delta w_{ij}(p) = \alpha \times x_i(p) \times \delta_j(p)$$
$$w_{ij}(p+1) = w_{ij}(p) + \Delta w_{ij}(p)$$

- **Bước 4*: Lặp***